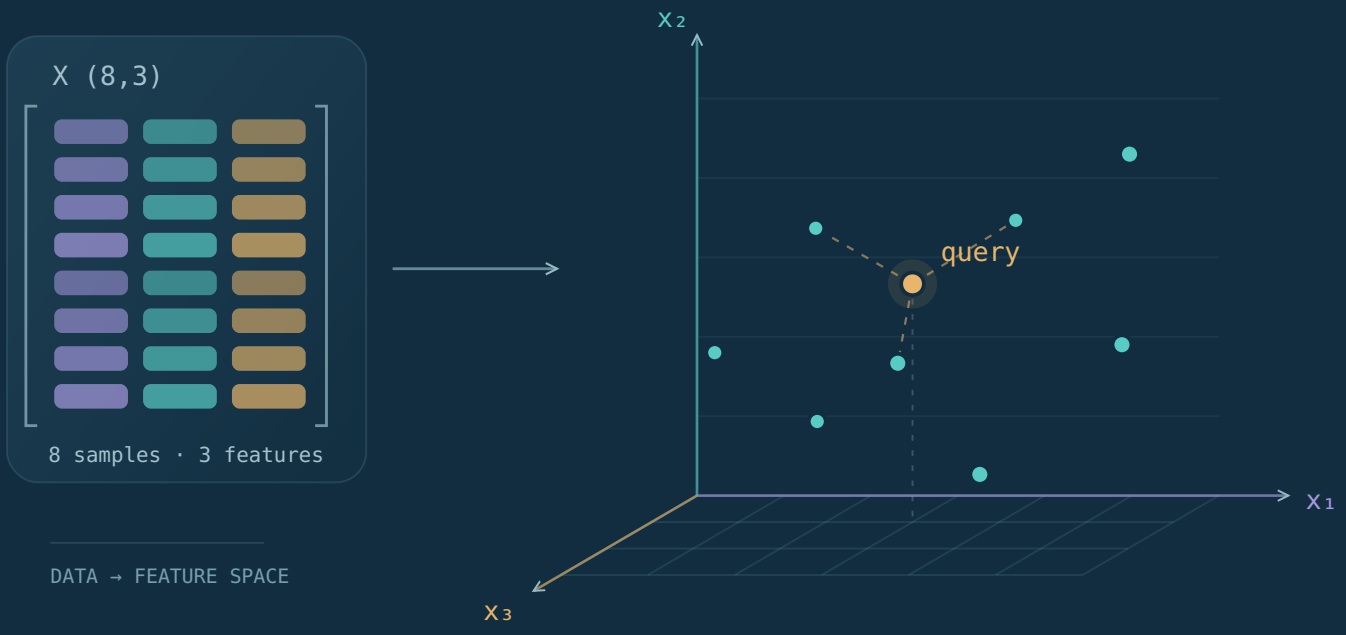
از یک جدول
تا نزدیک‌ترین نمونه

بردار، ماتریس و فاصلهٔ اقلیدسی

یک مسیر آموزشی با NumPy



همراه با مثال‌های مرحله‌به‌مرحله و تمرین

از خواندن داده تا محاسبهٔ فاصله و انتخاب همسایه‌ها

یادداشت آموزشی کلاس ماشین لرنینگ

از یک جدول تا نزدیک‌ترین نمونه

فهم بردار، ماتریس و فاصلهٔ اقلیدسی با NumPy

فرض کنید مشخصات چند دستگاه را داریم و می‌خواهیم بگوییم کدام‌یک به مشخصاتِ دلخواه ما نزدیک‌تر است. برای پاسخ، باید عبارتِ «نزدیک بودن» را به یک محاسبه تبدیل کنیم. این جزوه همین مسیر را دنبال می‌کند: داده را به صورت عددی می‌خوانیم، اختلاف دو نمونه را می‌سنجیم و سپس همان کار را برای همهٔ نمونه‌ها انجام می‌دهیم.

شناسه	قیمت	RAM	دوام باتری
A	50,100,000	8	3
B	52,000,000	16	10
C	35,000,000	8	6
D	80,000,000	32	12
E	60,000,000	16	8
F	45,000,000	16	7
G	55,000,000	16	12
H	58,000,000	24	9

این جدول، دادهٔ مرجع ما در تمام بحث است. هر سطر یک دستگاهِ فرضی و هر ستونِ عددی یک ویژگی است. ترتیب ویژگی‌ها را ثابت می‌گیریم: قیمت، RAM و دوام باتری. برای خواندن مثال، RAM را برحسب گیگابایت و باتری را برحسب ساعت تفسیر می‌کنیم؛ قیمت با همان واحد اولیهٔ داده نوشته شده است.

مشخصاتِ موردِ جست‌وجو را با یک بردارِ پرس‌وجو، یا `query`، بیان می‌کنیم. این بردار، یک دستگاهِ دیگر در جدول نیست؛ مرجعی است که هر سطر را با آن مقایسه خواهیم کرد.

$$\mathbf{q} = (50{,}000{,}000,\ 16,\ 10)$$

این سه عدد باید دقیقاً با همان ترتیبِ ستون‌های جدول خوانده شوند. جابه‌جا کردن RAM و باتری، معنای نمونه را تغییر می‌دهد؛ حتی اگر هنوز سه عدد در آرایه داشته باشیم. بنابراین بردارِ ویژگی، فقط مجموعه‌ای از عددها نیست؛ **یک فهرستِ مرتب از اندازه‌گیری‌های معنادار** است.

در پایان، شما نزدیک‌ترین نمونه و سه نمونهٔ نزدیک‌تر را پیدا می‌کنید. محاسباتِ نمونه‌های آموزشی را با عددهایی جدا از جدول انجام می‌دهیم تا نتیجهٔ تمرین اصلی برای حلِ خودتان باقی بماند.

برای وارد کردن جدول به NumPy، شناسه‌ها را از ویژگی‌های عددی جدا می‌کنیم. `names[i]` نامِ دستگاهی است که اطلاعاتش در `X[i]` قرار دارد. این هماهنگیِ اندیس‌ها را در تمام محاسبات نگه می‌داریم.

```
import numpy as np

names = np.array(["A", "B", "C", "D", "E", "F", "G", "H"])
X = np.array([
    [50_100_000,  8,  3],
    [52_000_000, 16, 10],
    [35_000_000,  8,  6],
    [80_000_000, 32, 12],
    [60_000_000, 16,  8],
    [45_000_000, 16,  7],
    [55_000_000, 16, 12],
    [58_000_000, 24,  9],
], dtype=float)

query = np.array([50_000_000, 16, 10], dtype=float)
```

به سطرِ اول نگاه کنید: یک نمونه با سه مؤلفه داریم. اگر هر ویژگی را یک محور در نظر بگیریم، این نمونه یک نقطه در فضای سه‌بُعدیِ ویژگی‌هاست. شناسهٔ A برای شناساییِ سطر است و در مختصاتِ این نقطه نقشی ندارد.

$$\mathbf{x}_A = (50{,}100{,}000,\ 8,\ 3) \in \mathbb{R}^3$$

حالا نگاه را از سطر به ستون تغییر دهید. ستونِ قیمت نیز یک بردار است، اما هشت مؤلفه دارد: قیمتِ هشت نمونه. بردارِ یک سطر، ویژگی‌های یک نمونه را توصیف می‌کند؛ بردارِ یک ستون، مقادیر یک ویژگی را در میان نمونه‌ها کنار هم می‌گذارد. کلِ داده، ماتریسی با هشت سطر و سه ستون است.

```
X.shape        # (8, 3): samples, features
X[0].shape     # (3,):   features of one sample
X[:, 0].shape  # (8,):   prices of all samples
```

در اینجا دو کاربردِ واژهٔ «بُعد» را باید از هم جدا کنیم. نمونهٔ `X[0]` سه مؤلفه دارد و در فضای ویژگیِ سه‌بُعدی است؛ ولی خودِ این آرایه فقط یک محور دارد و `X[0].ndim` برابر ۱ است. آرایهٔ `X` دو محور دارد، پس `X.ndim` برابر ۲ است. تعداد محورهای آرایه را با تعداد ویژگی‌های هر نمونه یکی نگیرید.

شکلِ `(3,)` یعنی یک محور با طول ۳؛ شکلِ `(1, 3)` یعنی دو محور، با یک سطر و سه ستون. در مثال‌های ما، هر سطرِ انتخاب‌شده آرایه‌ای با شکلِ `(3,)` است.

پیش از محاسبهٔ فاصله، باید بتوانیم بخشِ موردِ نظرِ داده را درست انتخاب کنیم. اندیس‌ها از صفر شروع می‌شوند. در `X[i, j]`، اندیسِ اول به نمونه و اندیسِ دوم به ویژگی مربوط است. علامت `:` تمامِ مواردِ آن محور را انتخاب می‌کند.

```
X[0, 1]        # RAM of the first sample
X[0]           # all features of the first sample
X[:, 0]        # prices of all samples
X[:3]          # first three samples
X[:, :2]       # first two features of all samples
```

وقتی ستونِ قیمت را جدا می‌کنیم، می‌توانیم دربارهٔ همین ویژگی سؤال کنیم: کمترین مقدار چیست؟ در کدام سطر است؟ میانگین چقدر است؟ پاسخِ سؤالِ اول یک مقدار است؛ پاسخِ سؤالِ دوم یک اندیس.

```
prices = X[:, 0]
lowest_price = np.min(prices)
cheapest_index = np.argmin(prices)
cheapest_name = names[cheapest_index]
average_price = prices.mean()
```

برای دیدنِ تفاوت، روی بردارِ مستقلِ `[9, 4, 7]`، کمینه برابر ۴ و اندیسِ کمینه برابر ۱ است. پس `argmin` نامِ نمونه یا خودِ کمینه را نمی‌دهد؛ موقعیت آن را می‌دهد. بعداً همین تفاوت برای انتخابِ نزدیک‌ترین نمونه مهم خواهد بود.

نوعِ دیگری از پرسش، انتخاب با شرط است. مثلاً می‌خواهیم دستگاه‌هایی را انتخاب کنیم که RAM آن‌ها حداقل ۱۶ است. مقایسهٔ زیر یک مقدارِ منطقی برای هر سطر می‌سازد؛ این بردارِ منطقی را ماسکِ بولی می‌نامیم.

```
ram_ok = X[:, 1] >= 16
number_ok = np.sum(ram_ok)
names_ok = names[ram_ok]
```

در جمعِ یک آرایهٔ بولی، `True` مانند ۱ و `False` مانند صفر شمرده می‌شود. بنابراین جمعِ ماسک، تعدادِ سطرهای پذیرفته‌شده را می‌دهد. با همین منطق، چند شرط را برای هر نمونه ترکیب می‌کنیم:

```
budget, min_ram, min_battery = 55_000_000, 16, 8
price_ok = X[:, 0] <= budget
ram_ok = X[:, 1] >= min_ram
battery_ok = X[:, 2] >= min_battery
eligible = price_ok & ram_ok & battery_ok
selected_names = names[eligible]
```

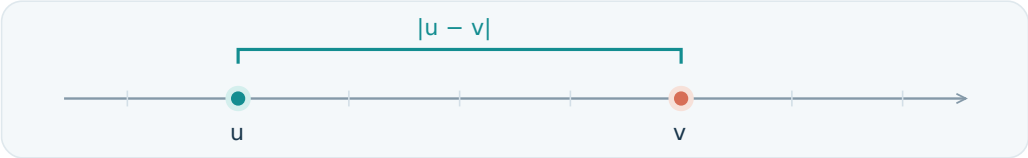
برای آرایه‌ها، `&` شرط‌ها را مؤلفه‌به‌مؤلفه ترکیب می‌کند؛ `and` جایگزینِ آن نیست. اگر مقایسه‌ها را در یک عبارت بنویسید، هر مقایسه باید داخل پرانتز باشد. همچنین `> 8` با `>= 8` فرق دارد: فقط دومی مقدارِ مرزیِ ۸ را می‌پذیرد.

اما سؤالِ اصلیِ ما هنوز پاسخ داده نشده است. شرط‌گذاری می‌گوید کدام سطرها محدودیت‌ها را رعایت می‌کنند؛ برای گفتنِ اینکه کدام سطر به یک پرس‌وجو شبیه‌تر است، به اندازه‌گیریِ فاصله نیاز داریم.

از اختلافِ دو مقدار تا فاصلهٔ دو نمونه

ابتدا فقط یک ویژگی را نگه دارید؛ مثلاً قیمت. در این نگاه، هر دستگاه یک عدد و در نتیجه یک نقطه روی خط است. فاصلهٔ دو نقطه، مقدارِ جداییِ آن‌هاست و نباید با تغییرِ ترتیبِ تفریق، منفی شود. پس قدرِ مطلقِ تفاضل را می‌گیریم:

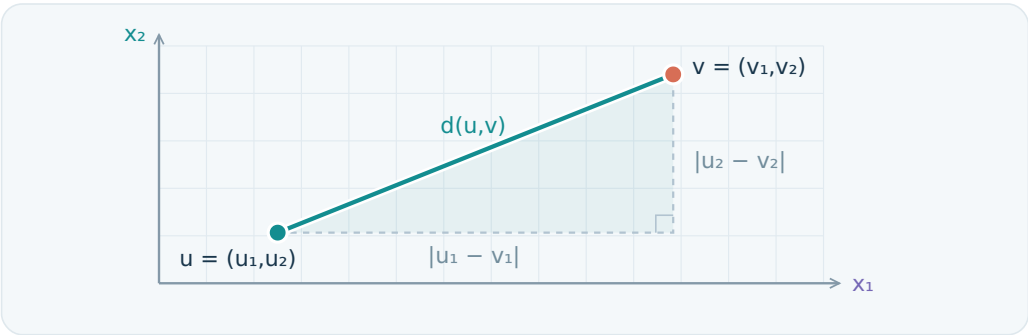
$$d(u, v) = |u - v| = \sqrt{(u - v)^2}$$



در NumPy، `np.abs` قدرِ مطلق را مؤلفه‌به‌مؤلفه محاسبه می‌کند. بنابراین می‌توان اختلافِ قیمتِ همهٔ دستگاه‌ها با قیمتِ پرس‌وجو را یک‌جا به فاصلهٔ یک‌بُعدی تبدیل کرد:

```
price_distances = np.abs(X[:, 0] - query[0])
```

این عبارت فقط شباهت از نظر قیمت را می‌سنجد. اگر RAM را هم وارد کنیم، هر نمونه دو مختصات خواهد داشت. اختلافِ دو نمونه روی هر محور، دو ضلعِ قائمِ یک مثلث را می‌سازد. کوتاه‌ترین مسیر میان نقاط، همان پاره‌خطِ مستقیم است؛ طولِ آن با رابطهٔ فیثاغورس به دست می‌آید.



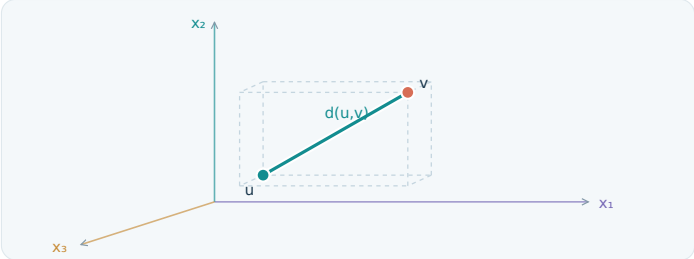
$$d(\mathbf{u}, \mathbf{v}) = \sqrt{(u_1 - v_1)^2 + (u_2 - v_2)^2}$$

مهم است که اختلاف‌ها را مستقیم جمع نکنیم. برای مثال، اختلافِ بردارهای `[1, 4]` و `[4, 1]` برابر `[-3, 3]` است. جمعِ اختلاف‌ها صفر می‌شود، در حالی که دو نقطه یکسان نیستند. توانِ دوم اجازه نمی‌دهد اختلاف‌های مثبت و منفی یکدیگر را خنثی کنند.

به این ترتیب، فاصله از یک دستورِ حفظی به یک ساختارِ قابلِ فهم تبدیل می‌شود: اختلافِ مختصات، مربعِ اختلاف‌ها، جمعِ آن‌ها و در پایان جذر. همین ساختار را در بُعدهای بیشتر نگه می‌داریم.

با اضافه شدنِ ویژگیِ سوم، هر نمونه در فضای سه‌بُعدی قرار می‌گیرد. در دادهٔ ما این مؤلفهٔ سوم، دوام باتری است. برای فاصله، سهمِ این محور هم باید واردِ مجموعِ مربع‌ها شود:

$$d(\mathbf{u}, \mathbf{v}) = \sqrt{(u_1 - v_1)^2 + (u_2 - v_2)^2 + (u_3 - v_3)^2}$$



شکل، فقط یک نمایشِ مفهومی از دو نقطه در سه بُعد است و جایگاهِ واقعیِ دستگاه‌های جدول را نشان نمی‌دهد. وقتی تعدادِ ویژگی‌ها از سه بیشتر شود، ترسیم مستقیم دشوار است؛ ولی محاسبه تغییری نمی‌کند. برای دو بردار با n مؤلفه، داریم:

$$\mathbf{u}, \mathbf{v} \in \mathbb{R}^n, \qquad d(\mathbf{u}, \mathbf{v}) = \sqrt{\sum_{j=1}^{n} (u_j - v_j)^2}$$

اندیسِ j ویژگی‌ها را می‌شمارد؛ بنابراین n تعدادِ مؤلفه‌های هر نمونه است، نه تعداد نمونه‌ها. دو بردار باید تعدادِ مؤلفه‌های یکسان و ویژگی‌های متناظر داشته باشند؛ مثلاً قیمت از قیمت کم شود و واحدِ قیمت در هر دو یکسان باشد.

اکنون همین فرمول را برای دو بردارِ مستقل از جدول، خط‌به‌خط به NumPy تبدیل کنیم. در هر مرحله، معنای خروجی روشن است:

```
u = np.array([1, 2, 3], dtype=float)
v = np.array([4, 6, 3], dtype=float)

difference = u - v             # [-3., -4., 0.]
squared = difference ** 2      # [9., 16., 0.]
sum_squared = np.sum(squared)  # 25.
distance = np.sqrt(sum_squared)  # 5.
```

تفریق و توانِ دوم هنوز یک بردارِ سه‌مؤلفه‌ای می‌دهند؛ با جمع، مؤلفه‌ها به یک عدد تبدیل می‌شوند و جذرِ آن عدد، فاصله است. چون تنها دو نقطه را مقایسه می‌کنیم، خروجی یک عدد است. علامتِ توان در پایتون `**` است؛ `^` عملگرِ توان نیست. همان چهار گام را می‌توان در یک عبارت هم نوشت:

```
distance = np.sqrt(np.sum((u - v) ** 2))
```

همان محاسبه، برای همهٔ سطرها

حالا به پرس‌وجوی اصلی برگردیم. اگر فقط فاصلهٔ نمونهٔ اول را بخواهیم، `X[0] - query` سه اختلاف می‌دهد. برای همهٔ نمونه‌ها باید همین تفریق را با تک‌تکِ سطرها انجام دهیم. NumPy اجازه می‌دهد آن را بدون نوشتنِ حلقهٔ صریح بیان کنیم:

```
differences = X - query
# X.shape:           (8, 3)
# query.shape:          (3,)
# differences.shape:  (8, 3)
```

این رفتار، broadcasting نام دارد. در این مثال، بردارِ سه‌مؤلفه‌ای با ستون‌های ماتریس تطبیق پیدا می‌کند و از هر سطر کم می‌شود. در نتیجه، هر سطرِ `differences` اختلافِ یک دستگاه با پرس‌وجو است. این «تکرار در سطرها» توضیحِ مفهومیِ عملیات است؛ لازم نیست خودمان از پرس‌وجو هشت کپی بسازیم.

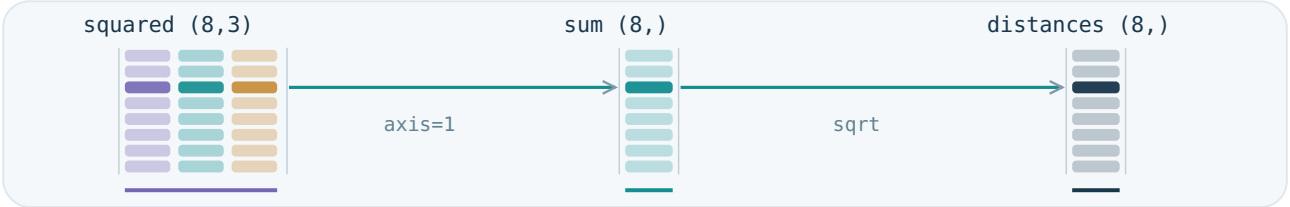
قاعدهٔ شکل‌ها از سمتِ راست بررسی می‌شود: طولِ محورهای متناظر باید برابر باشد یا یکی از آن‌ها ۱ باشد؛ محورِ غایب نیز طولِ ۱ در نظر گرفته می‌شود. در اینجا محورِ آخر در هر دو ورودی طولِ ۳ دارد و شکلِ `(3,)` برای این عملیات مانند `(1, 3)` با سطرها هماهنگ می‌شود.

توانِ دوم را می‌توانیم بلافاصله اعمال کنیم؛ چالشِ بعدی، جمع است. اکنون به‌جای یک بردار، ماتریسی از مربعِ اختلاف‌ها داریم. اگر همهٔ عناصر را با هم جمع کنیم، اطلاعاتِ هشت نمونه در یک عدد ادغام می‌شود. ما یک مجموع برای هر سطر می‌خواهیم. برای دیدنِ تفاوت، مثالِ کوچکِ زیر را بخوانید:

```
M = np.array([[1, 2, 3],
              [4, 5, 6]])
np.sum(M)          # 21: all elements
np.sum(M, axis=0)  # [5, 7, 9]: one sum per column
np.sum(M, axis=1)  # [6, 15]: one sum per row
```

محورِ صفر در دادهٔ ما محورِ نمونه‌ها و محورِ یک محورِ ویژگی‌هاست. با `axis=1`، سه ویژگیِ داخلِ هر سطر را جمع می‌کنیم؛ محورِ ویژگی‌ها حذف می‌شود و هشت مقدار باقی می‌ماند. سپس از هر یک جذر می‌گیریم:

```
squared = differences ** 2             # (8, 3)
sum_squared = np.sum(squared, axis=1)  # (8,)
distances = np.sqrt(sum_squared)      # (8,)
```



پس `distances[i]` به همان دستگاهی تعلق دارد که در `X[i]` و `names[i]` قرار دارد. درستیِ این تطبیق را پیش از هر انتخاب و مرتب‌سازی نگه دارید. اکنون فاصلهٔ همهٔ نمونه‌ها را داریم؛ اما پیش از انتخابِ کمترین فاصله، باید ببینیم این عددها دقیقاً چه چیزی را اندازه می‌گیرند.

در فرمولِ اقلیدسی، هیچ نامی از «قیمت» یا «RAM» وجود ندارد؛ فقط اختلافِ عددها واردِ محاسبه می‌شود. در دادهٔ ما، قیمت‌ها در حدِ ده‌ها میلیون‌اند، در حالی که دو ویژگیِ دیگر عددهای بسیار کوچک‌تری دارند. بنابراین اختلافِ قیمت می‌تواند سهمِ دو ویژگیِ دیگر را در مجموعِ مربع‌ها تحت‌الشعاع قرار دهد.

برای روشن شدنِ موضوع، از جدول فاصله بگیریم: اختلافِ ۲ میلیون در قیمت، پس از توانِ دوم به ۴ تریلیون تبدیل می‌شود؛ اختلافِ ۴ واحدیِ RAM فقط ۱۶ به مجموع اضافه می‌کند. این تفاوت، از معنای اهمیتِ ویژگی‌ها نیامده است؛ از مقیاسِ عددیِ آن‌ها آمده است. بنابراین «نزدیک‌ترین» بدون مشخص کردنِ نمایش و مقیاسِ ویژگی‌ها، توصیفِ کاملی نیست.

یک آزمایشِ ساده انجام می‌دهیم: قیمت را به‌جای واحدِ اولیه، برحسب میلیون بیان می‌کنیم و دو ویژگیِ دیگر را همان‌طور نگه می‌داریم. باید هم ستونِ قیمتِ داده و هم مؤلفهٔ قیمتِ پرس‌وجو تبدیل شوند؛ در غیر این صورت، دو مقدار با واحدهای متفاوت از یکدیگر کم می‌شوند.

```
X_million = X.copy()
query_million = query.copy()

X_million[:, 0] /= 1_000_000
query_million[0] /= 1_000_000

distances_million = np.sqrt(
    np.sum((X_million - query_million) ** 2, axis=1)
)
```

از `copy()` استفاده کردیم تا آرایه‌های اولیه برای مقایسه باقی بمانند. انتسابِ سادهٔ `X_million = X` کپیِ مستقل نمی‌سازد؛ دو نام به یک آرایه اشاره می‌کنند و تغییرِ عناصر از طریقِ نامِ جدید، دادهٔ اولیه را هم تغییر می‌دهد.

اگر اختلافِ قیمت، RAM و باتری را به‌ترتیب با نمادهای زیر نشان دهیم، فاصله در نمایشِ تازه چنین می‌شود:

$$d_{\text{million}} = \sqrt{\left(\frac{\Delta p}{10^6}\right)^2 + (\Delta r)^2 + (\Delta b)^2}$$

بنابراین سهمِ مربعِ اختلافِ قیمت بر یک تریلیون تقسیم شده است. این تبدیل می‌تواند ترتیبِ فاصله‌ها را تغییر دهد؛ چون فقط یک ویژگی تغییر مقیاس داده است. اگر همهٔ مؤلفه‌ها با یک ضریبِ مثبتِ یکسان ضرب شوند، فاصله‌ها نیز با همان ضریب تغییر می‌کنند و ترتیبشان باقی می‌ماند، اما اینجا چنین کاری نکرده‌ایم.

بیانِ قیمت برحسب میلیون، استانداردسازیِ ویژگی‌ها نیست و تضمین نمی‌کند سه ویژگی اهمیتِ برابر پیدا کرده باشند. فعلاً هدف، دیدنِ اثرِ مقیاس است. در گزارشِ نتیجه باید روشن کنیم فاصله را با کدام نمایش حساب کرده‌ایم. از خودتان بپرسید: با تبدیلِ قیمت، چه سهمی در فرمول تغییر کرده و چرا ممکن است همسایه‌های نزدیک عوض شوند؟

اکنون می‌توانیم از فاصله‌ها به انتخاب برسیم. کوچک‌ترین فاصله، بیشترین شباهت به پرس‌وجو را طبقِ همین معیار نشان می‌دهد. این شباهت به‌تنهایی به معنی بهترین دستگاه نیست؛ مثلاً یک باتریِ بسیار قوی‌تر از مقدارِ پرس‌وجو هم از نظر عددی با آن اختلاف دارد.

در مرورِ قیمت‌ها دیدیم که `min` مقدارِ کمینه و `argmin` اندیسِ آن را می‌دهد. اینجا نیز به اندیس نیاز داریم تا نام و فاصله را از آرایه‌های هماهنگ انتخاب کنیم. ابزار را ابتدا روی چهار فاصلهٔ فرضی، مستقل از تمرینِ اصلی، ببینید:

```
labels_demo = np.array(["P", "Q", "R", "S"])
d_demo = np.array([7.5, 2.0, 9.0, 4.5])

i_demo = np.argmin(d_demo)
labels_demo[i_demo]  # 'Q'
d_demo[i_demo]       # 2.0
```

اگر چند همسایه بخواهیم، فقط پیدا کردنِ کمینه کافی نیست. باید اندیسِ فاصله‌ها را به ترتیبِ صعودی به دست آوریم و تعدادی از ابتدای آن را برداریم. `np.argsort` همین ترتیبِ اندیس‌ها را می‌دهد:

```
order_demo = np.argsort(d_demo)  # [1, 3, 0, 2]

k = 2
chosen_demo = order_demo[:k]
labels_demo[chosen_demo]  # ['Q', 'S']
d_demo[chosen_demo]       # [2.0, 4.5]
```

این اندیس‌ها مانند نخِ اتصالِ نام‌ها و فاصله‌ها هستند. اگر فقط خودِ فاصله‌ها را با `np.sort` مرتب کنیم، مقدارهای مرتب‌شده را داریم، اما از آن خروجی به‌تنهایی نمی‌توانیم موقعیتِ اولیهٔ نمونه‌ها را بخوانیم. پس برای انتخابِ شناسه و سطر و فاصله، از یک مجموعه اندیسِ یکسان استفاده می‌کنیم.

اگر چند فاصله برابر باشند، ممکن است انتخابِ «یک» همسایه یکتا نباشد. `argmin` اولین اندیسِ کمینه را برمی‌گرداند. ترتیبِ مواردِ برابر در مرتب‌سازیِ پیش‌فرض را تضمین‌شده فرض نکنید و اگر تساوی دیدید، در گزارش آن را ذکر کنید.

مسیرِ حل اکنون کامل است: هر سطر را به عنوانِ یک بردار بخوانید، مؤلفه‌های متناظرِ پرس‌وجو را از آن کم کنید، مربع‌ها را درونِ هر سطر جمع بزنید، جذر بگیرید و با اندیس‌ها انتخاب کنید. این مراحل وابسته به داشتنِ دقیقاً سه ویژگی نیستند؛ اگر تعداد ویژگی‌ها بیشتر شود، طولِ سطر تغییر می‌کند و منطقِ محاسبه همان است.

پیش از رفتن به تمرین، بدون اجرای کد این سؤال را جواب دهید: اگر فقط ترتیبِ نزدیک بودن را بخواهیم، آیا می‌توانیم جذر را حذف کنیم؟ به این فکر کنید که تابعِ جذر روی عددهای نامنفی صعودی است. سپس توضیح دهید چرا این استدلال به معنی برابر بودنِ «مجموعِ مربع‌ها» با خودِ «فاصله» نیست.

حالا شما مسیر را کامل کنید

با همان هشت سطرِ داده و پرس‌وجوی زیر کار کنید. پرس‌وجو را به سطرهای داده اضافه نکنید و فیلترِ بودجه، RAM یا باتری را روی این تمرین اعمال نکنید؛ مقایسه باید با همهٔ هشت نمونه انجام شود.

```
query = np.array([50_000_000, 16, 10], dtype=float)
```

1. **محاسبه را پیش از اجرا پیش‌بینی کنید.** شکلِ خروجیِ تفریق، توانِ دوم، جمعِ سطری و جذر را بنویسید. توضیح دهید چرا تعدادِ فاصله‌ها باید برابرِ تعدادِ نمونه‌ها باشد.

2. **فاصله‌ها را بسازید.** محاسبه را ابتدا در چهار خطِ جدا، با تفریق، توانِ دوم، جمع با محورِ مناسب و جذر بنویسید. برای کنترل، فاصلهٔ یکی از سطرها را جداگانه محاسبه و با مؤلفهٔ متناظرِ بردارِ فاصله‌ها مقایسه کنید.

3. **همسایه‌ها را انتخاب کنید.** نزدیک‌ترین نمونه و سه نمونهٔ نزدیک‌تر را به ترتیبِ فاصله پیدا کنید. همراهِ هر شناسه، مقدارِ فاصله را نیز گزارش کنید.

4. **اثرِ مقیاس را بررسی کنید.** با حفظِ دادهٔ اولیه، قیمت را در داده و پرس‌وجو برحسب میلیون بیان کنید و مرحلهٔ قبل را تکرار کنید. توضیح دهید آیا رتبه‌بندی تغییر کرده و تغییر را چطور با ساختارِ فرمول مرتبط می‌دانید.

نمایشِ قیمت	نزدیک‌ترین نمونه و فاصله	سه نمونه، از نزدیک به دور
واحدِ اولیه	.....	.....
برحسب میلیون	.....	.....

پاسخِ شما باید شاملِ کدِ قابلِ اجرا، نتیجهٔ هر دو مقیاس و چند جمله دربارهٔ منطقِ محاسبه باشد. برای اطمینان، بررسی کنید که فاصله‌ها نامنفی‌اند، نام‌ها و فاصله‌ها با اندیس‌های یکسان انتخاب شده‌اند و فهرستِ سه همسایه از فاصلهٔ کوچک‌تر به بزرگ‌تر مرتب شده است. اگر تساوی وجود داشت، آن را مشخص کنید.

این جزوه پاسخِ عددی یا شناسه‌های نهاییِ تمرین را ارائه نمی‌کند. مثال‌های حل‌شدهٔ میانی فقط برای فهمِ ابزارها و مراحلِ استدلال‌اند.

برای مراجعهٔ بیشتر به مستندات رسمی NumPy:

Indexing · Broadcasting · numpy.sum · numpy.argsort